IBM GENERATIVE AI MINI PROJECT

AI PERSONAL STUDY ASSISTANT

Python Programming for Beginners

STUDY MATERIAL
A practical guide for first-year CSE (AI & ML) students

Submitted by	Muhammad ansal a
Department	Computer Science and Engineering (AI & ML)
College	TKMIT
Course	IBM Generative AI Mini Project

Learn the fundamentals. Practise with examples. Build your first AI-ready skills.

PAGE 2 — Introduction to Python

1.1 What is Python?

Python is a high-level, general-purpose programming language known for readable syntax and ease of learning. A programming language lets people give instructions to computers. Python is used by students, software developers, data analysts, researchers, and AI engineers.

```
print("Hello, World!")
```

Expected output

```
Hello, World!
```

The print() function displays information on the screen.

1.2 History and development

Python was created by Dutch programmer Guido van Rossum. Development began in the late 1980s, and Python was first publicly released in 1991. It emphasizes readability and developer productivity. The name was inspired by the comedy group Monty Python. Python 3, introduced in 2008, is the modern version used for most new projects.

1.3 Key features and advantages

Feature	Explanation
Easy to learn	Readable, straightforward syntax.
Interpreted	Executed by the Python runtime.
Cross-platform	Runs on Windows, macOS, Linux, and other supported platforms.
Open-source	Available under its license for use and distribution.
Large library ecosystem	Tools for data science, AI, web development, and more.
Dynamically typed	Types are determined at runtime.
Object-oriented	Supports classes, objects, and inheritance.

1.4 Real-world applications

AI and machine learning: chatbots, recommendation systems, image classifiers, and predictive models.

Data science: analyzing student marks, research data, and business trends.

Web development: websites, APIs, and web applications.

Automation and science: processing reports, renaming files, calculations, and experiments.

1.5 Why Python is useful for AI

AI is the field of building systems that perform tasks associated with abilities such as reasoning, language understanding, or perception. Machine learning (ML) is a part of AI in which computers learn patterns from data to make predictions or decisions.

Libraries support mathematical calculations and data processing.

Tools help create, train, and evaluate machine learning models.

Graphs and charts help explain data and results.

Python is widely used in AI education, research, and experimentation.

Student-life example: Python can organize examination marks, calculate averages, visualize trends, and help build a model to identify students who may need academic support.

Quick exercise: Write statements that display your name and department.

PAGE 3 — Variables and Data Types

2.1 What is a variable?

A variable is a name that refers to a value. A student system might store a student's name, age, semester, and marks in variables.

```
student_name = "Anu"
age = 18
marks = 87.5
is_present = True

print(student_name)
print(age)
print(marks)
print(is_present)
```

Expected output

```
Anu
18
87.5
True
```

The = symbol assigns a value to a variable; it does not mean mathematical equality in an expression.

2.2 Rules for naming variables

Begin with a letter or underscore; do not begin with a number.

Use letters, numbers, and underscores. Spaces and hyphens are not allowed.

Names are case-sensitive: age and Age are different.

Do not use Python keywords such as if, for, or class.

Prefer descriptive names such as total_marks.

Valid	Invalid
student_name	student name
age1	1age
_total	total-marks
college_name	class

2.3 Basic data types

Type	Meaning	Example
int	Whole numbers	age = 18
float	Numbers with decimal points	height = 1.72
str	Text	name = "Anu"
bool	True or False	is_passed = True

```
age = 18
percentage = 87.5
name = "Anu"
is_passed = True

print(type(age))
print(type(percentage))
print(type(name))
print(type(is_passed))
```

Expected output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

2.4 Type conversion

Type conversion changes a value from one data type to another when valid. Common functions include int(), float(), str(), and bool().

```
age_text = "18"
age = int(age_text)
```

```
price = 25
price_text = str(price)

print(age + 2)
print(price_text)
print(float("3.5"))
```

Expected output

```
20
25
3.5
```

The string "18" becomes an integer, so arithmetic can be performed. int("hello") raises ValueError because the text is not a valid integer.

2.5 Taking input

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello", name)
print("Next year, you will be", age + 1)
```

Expected output

```
Example input: Anu, then 18
Hello Anu
Next year, you will be 19
```

Practice: Create variables for your name, age, and favourite subject. Convert the string "100" to an integer and multiply it by 2.

PAGE 4 — Operators and Expressions

3.1 What are operators?

Operators are symbols or keywords that perform operations on values and variables. An expression combines values, variables, and operators to produce a result. For example, 10 + 5 is an expression and + is an operator.

3.2 Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3.333...
//	Floor division	10 // 3	3
%	Remainder	10 % 3	1
**	Exponentiation	2 ** 3	8

```
a = 10
b = 3
print(a + b)
print(a // b)
print(a % b)
print(2 ** 3)
```

Expected output

```
13
3
1
8
```

3.3 Comparison operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	8 > 4	True
<	Less than	2 < 1	False
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	3 <= 4	True

```
marks = 75
print(marks >= 40)
print(marks == 100)
print(marks < 50)
```

Expected output

```
True
False
False
```

3.4 Logical operators

Operator	Meaning
and	True when both conditions are true.
or	True when at least one condition is true.
not	Reverses a Boolean value.

```
marks = 85
attendance = 90
print(marks >= 40 and attendance >= 75)
print(marks < 40 or attendance < 75)
print(not (marks < 40))
```

Expected output

```
True
False
True
```

3.5 Assignment and precedence

Operator	Example	Equivalent
=	x = 5	Assign 5 to x
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2

```
score = 10
score += 5
score *= 2
print(score)
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Expected output

```
30
14
20
```

Remember: = assigns a value, while == compares values. Parentheses can make the intended order of calculations clear.

Practice: Calculate the total cost of three notebooks costing ₹25 each and test whether the total exceeds ₹100.

PAGE 5 — Conditional Statements and Loops

4.1 Conditional statements

Conditions let a program choose which instructions to run.

```
marks = 80
if marks >= 40:
    print("You passed!")
```

Expected output

```
You passed!
```

An if block runs when its condition is true. An else block runs when the if condition is false.

```
number = 7
if number % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

Expected output

```
Odd number
```

Use elif to test additional conditions. Python uses indentation to identify blocks; four spaces per level is a common convention.

```
marks = 76
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Needs improvement")
```

Expected output

```
Grade B
```

4.2 The for loop

A for loop repeats a block for each item in a sequence or other iterable.

```
for i in range(1, 6):
    print(i)
```

Expected output

```
1
2
3
4
5
```

range(1, 6) produces 1 through 5; the ending value 6 is excluded.

4.3 Multiplication table

```
number = 5
for i in range(1, 6):
    print(number, "x", i, "=", number * i)
```

Expected output

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
```

4.4 The while loop

A while loop repeats as long as its condition remains true. Update the condition's controlling variable to avoid an unintended infinite loop.

```
count = 1
while count <= 3:
    print("Study session", count)
    count += 1
```

Expected output

```
Study session 1
Study session 2
Study session 3
```

4.5 break and continue

break immediately terminates the nearest enclosing loop. continue skips the rest of the current iteration and moves to the next one.

```
for i in range(1, 6):
    if i == 4:
        break
    print(i)

for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Expected output

```
1
2
3


1
2
4
5
```

4.6 Practical example: attendance

```
attendance = 82
if attendance >= 75:
    print("Attendance requirement met")
else:
    print("Attendance requirement not met")
```

Expected output

```
Attendance requirement met
```

Practice: Check whether a number is positive, negative, or zero. Print 1–10 with a while loop, then print even numbers from 1 to 20.

PAGE 6 — Functions and Data Structures

5.1 What is a function?

A function is a reusable block of code designed to perform a task. Functions reduce repetition and help organize, test, and maintain programs. Python includes print(), input(), and len(), and you can define your own functions with def.

Parameters, arguments, and return values

Parameter: a variable listed in a function definition.

Argument: a value supplied when the function is called.

Return value: a result sent back using return.

```
def add_numbers(a, b):
    return a + b

result = add_numbers(10, 20)
print(result)
```

Expected output

```
30
```

Here, a and b are parameters; 10 and 20 are arguments; 30 is the return value.

```
def greet(name):
    print("Welcome,", name)

greet("Anu")
```

Expected output

```
Welcome, Anu
```

5.2 Lists

A list is an ordered, mutable collection. It is useful for subjects, marks, tasks, and shopping items.

```
subjects = ["Python", "Mathematics", "Physics"]
subjects.append("English")
print(subjects)
print(subjects[0])
```

Expected output

```
['Python', 'Mathematics', 'Physics', 'English']
Python
```

5.3 Tuples

A tuple is an ordered collection that cannot be modified after creation. It is useful for fixed coordinates or RGB colour values.

```
coordinates = (10, 20)
print(coordinates)
print(coordinates[1])
```

Expected output

```
(10, 20)
20
```

5.4 Sets

A set stores unique elements. It removes duplicates and does not guarantee a fixed display order.

```
numbers = {1, 2, 2, 3, 3, 4}
print(sorted(numbers))
```

Expected output

```
[1, 2, 3, 4]
```

5.5 Dictionaries

A dictionary stores key-value pairs. Keys let you look up their associated values.

```
student = {"name": "Anu", "age": 18, "marks": 92}
print(student["name"])
print(student["marks"])
```

Expected output

```
Anu
92
```

5.6 Compare data structures

Feature	List	Tuple	Set	Dictionary
Structure	Ordered sequence	Ordered sequence	Unique elements	Key-value pairs
Mutable?	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not retained	Keys are unique
Access	Index	Index	Membership	Key
Use	Changing collections	Fixed collections	Unique values	Named records

Practice: Create a dictionary with a student's name, course, and marks. Store three subjects in a list and print the first subject.

PAGE 7 — Object-Oriented Programming

6.1 What is OOP?

Object-oriented programming (OOP) organizes code around objects that combine data and behaviour. For a student, the name and semester are data, while displaying a profile or updating the semester are actions.

6.2 Classes and objects

A class is a blueprint describing the attributes and behaviours its objects can have. An object is an individual instance of a class. Student can be a class; Anu and Rahul can be separate student objects.

6.3 Constructors, attributes, and methods

Constructor: __init__() is commonly used to initialize a new object.

Attribute: a variable associated with an object or class.

Method: a function defined inside a class.

self: refers to the current instance.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

student1 = Student("Anu", 90)
student1.display()
```

Expected output

```
Name: Anu
Marks: 90
```

6.4 Inheritance

Inheritance lets a class reuse or extend behaviour from another class. The original class is the parent/base class; the class that inherits is the child/derived class.

```
class Person:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print("My name is", self.name)

class Student(Person):
    def study(self):
        print(self.name, "is studying Python")

student = Student("Anu")
student.introduce()
student.study()
```

Expected output

```
My name is Anu
Anu is studying Python
```

Student inherits introduce() from Person and adds its own study() method.

6.5 Encapsulation

Encapsulation groups data and methods together while managing access to data. Python naming conventions and properties can help control access. A name beginning with two underscores triggers name mangling; it is not absolute security.

```
class Student:
    def __init__(self, marks):
        self.__marks = marks

    def get_marks(self):
        return self.__marks
```

```
student = Student(85)
print(student.get_marks())
```

Expected output

```
85
```

The get_marks() method provides a controlled way to retrieve the marks.

6.6 Where is OOP useful?

OOP is useful in student management systems, banking applications, games, inventory software, and larger AI applications. A college application might define Student, Teacher, Course, and AttendanceRecord classes.

Practice: Create a Book class with a title attribute and a method that displays it. Explain the difference between a class and an object.

PAGE 8 — Python for Artificial Intelligence

7.1 Key AI concepts

Concept	Simple explanation	Example
Artificial Intelligence	The broader field of intelligent computer systems	Conversational assistant
Machine Learning	Methods that learn patterns from data	Predicting house prices
Deep Learning	ML using multi-layer neural networks	Image recognition
Generative AI	AI that creates new content from learned patterns	Generating explanations or summaries

Machine learning is a subset of AI, and deep learning is a subset of machine learning. Generative AI overlaps with these areas and is often built using deep learning.

7.2 A typical AI workflow

Step	What happens
1. Collect data	Gather marks, images, text, or sensor readings.
2. Prepare data	Correct errors and organize the information.
3. Train a model	Use example data to learn patterns.
4. Evaluate and use	Check performance and make predictions.

Data quality and suitability strongly affect results. Models should be evaluated before being used in real decisions.

7.3 Common Python libraries

Library	Main purpose	Example use
NumPy	Numerical computing and multidimensional arrays	Calculating averages across numeric data
Pandas	Cleaning, organizing, and analyzing tables	Analyzing marks from a spreadsheet
Matplotlib	Creating charts and graphs	Plotting subject-wise marks
Scikit-learn	Common machine learning tasks and evaluation	Classification, regression, clustering

7.4 Analyze student marks with basic Python

This example performs simple data analysis; it is not machine learning by itself.

```
marks = [78, 85, 92, 67, 88]
average = sum(marks) / len(marks)

print("Marks:", marks)
print("Average:", round(average, 2))
print("Highest mark:", max(marks))
print("Lowest mark:", min(marks))
```

Expected output

```
Marks: [78, 85, 92, 67, 88]
Average: 82.0
Highest mark: 92
Lowest mark: 67
```

7.5 Visualize data with Matplotlib

```
import matplotlib.pyplot as plt

subjects = ["Python", "Maths", "Physics"]
marks = [85, 90, 78]

plt.bar(subjects, marks)
plt.title("Subject-wise Marks")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.show()
```

Expected result: a bar chart titled Subject-wise Marks with bars for Python (85), Maths (90), and Physics (78). The chart appears in a graphical output window or notebook.

7.6 Install the libraries

```
python -m pip install numpy pandas matplotlib scikit-learn
```

Run the command in a terminal with the intended Python environment active. Package installation requires a working package manager and environment.

Practice: Add two marks to the dataset, calculate the new average, and explain why a chart can make comparisons easier.

PAGE 9 — Practice Questions and Mini Quiz

Try answering the questions before checking the answer key. The quiz covers programming fundamentals, collections, OOP, and AI.

Section A — Multiple choice

1. Who created Python?
A. James Gosling B. Guido van Rossum C. Dennis Ritchie D. Tim Berners-Lee

2. Which data type stores a whole number?
A. str B. float C. int D. bool

3. What is the output of print(10 % 3)?
A. 3 B. 1 C. 3.33 D. 0

4. Which loop is commonly used to iterate over a sequence?
A. for B. if C. class D. return

5. Which collection stores key-value pairs?
A. List B. Tuple C. Set D. Dictionary

Section B — Short answers

6. What is a variable? Give one example.

7. Explain the difference between a list and a tuple.

8. What are a class and an object? Explain with a real-world example.

Section C — Coding exercises

9. Even or odd: Write a program that checks whether a number entered by the user is even or odd.

10. Student marks: Write a function that accepts three marks and returns their average. Display the average and print Pass if it is at least 40; otherwise print Fail.

Answer key with brief explanations

Q	Answer	Explanation
1	B — Guido van Rossum	He created Python.
2	C — int	Represents whole numbers such as 5 and 100.
3	B — 1	% returns the remainder after division.
4	A — for	Iterates over sequences and other iterables.
5	D — Dictionary	Associates keys with values.
6	Example: age = 18	A variable is a name that refers to a value.
7	Lists are mutable; tuples are immutable.	Both are ordered collections.
8	Class = blueprint; object = instance.	Student can be a class; one student's record can be an object.

Sample solution for Question 9

```
number = int(input("Enter a number: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Expected output

```
Example input: 8
Even
```

Sample solution for Question 10

```
def average(a, b, c):
    return (a + b + c) / 3

result = average(70, 80, 90)
print("Average:", result)
if result >= 40:
    print("Pass")
else:
    print("Fail")
```

Expected output

```
Average: 80.0
Pass
```

Mini-project challenge: Build a Student Study Tracker that stores a student's name and subjects, records study hours, calculates total study time, and displays a message based on the total.

PAGE 10 — Summary and References

9.1 Summary of important concepts

Topic	Main concept	Example
Variables	Names that refer to values	age = 18
Data types	Kinds of values	int, str, float, bool
Operators	Calculations and comparisons	a + b; a >= b
Conditions	Make decisions	if, elif, else
Loops	Repeat instructions	for, while
Functions	Reusable blocks of code	def add(a, b):
Data structures	Organize collections	list, tuple, set, dict
OOP	Organize code around objects	class, inheritance
AI libraries	Analyze data and build models	NumPy, Pandas, Matplotlib, Scikit-learn

9.2 Key takeaways for beginners

Learn the fundamentals before moving to advanced AI topics.
Practise writing code instead of only reading examples.
Understand errors and debug them systematically.
Break large problems into smaller functions.
Choose data structures that fit the task.
Use real-world datasets to practise analysis.
Evaluate machine learning models and pay attention to data quality.
Build small projects to turn theory into practical skills.

9.3 Glossary

Term	Definition
Algorithm	A sequence of steps to solve a problem.
Variable	A name that refers to a value.
Expression	Values and operators that produce a result.
Iteration	One repetition of a loop.
Parameter / Argument	A parameter appears in a function definition; an argument is supplied in a call.
Mutable / Immutable	Able / unable to be changed after creation.
Class / Object	A blueprint / an instance of a class.
Inheritance	Deriving a class from another class.
Encapsulation	Grouping data and behaviour while managing access.
Dataset	A collection of related data.
Model	A representation used to perform a task or make predictions.
Training	Fitting a machine learning model to data.
Library	Reusable code providing functions and tools.

9.4 Reliable learning resources

Python Official Documentation — Reference for Python syntax, built-in functions, and language features.
https://docs.python.org/3/

W3Schools Python Tutorial — Beginner explanations, examples, and practice.
https://www.w3schools.com/python/

NumPy Documentation — Arrays, numerical operations, and scientific computing.
https://numpy.org/doc/

Pandas Documentation — Cleaning, transforming, and analyzing tabular data.
https://pandas.pydata.org/docs/

Matplotlib Tutorials — Charts, graphs, and data visualization.
https://matplotlib.org/stable/tutorials/

Scikit-learn User Guide — Machine learning workflows, algorithms, and evaluation.
https://scikit-learn.org/stable/user_guide.html

9.5 Conclusion

Python provides a strong foundation for students beginning their Computer Science and Artificial Intelligence journey. Variables, operators, conditions, loops, functions, data structures, and OOP support the creation of useful applications. Python libraries extend these skills into data analysis, visualization, and machine learning.

This AI Personal Study Assistant material can serve as a knowledge base for a document-based chatbot. Students can ask questions, review code examples, and practise concepts at their own pace.

PDF preparation reminder: replace [Your Name] and [Your College Name] on the title page. Review page layout after export and keep code/output blocks together where possible.